Organização e Arquitetura de Computadores

Memória

Quelita Ribeiro

Departamento de Computação / UFS

Roteiro

- ✓Ordenação de bytes
- ✓ Códigos de correção de erro
- ✓Discos magnéticos

Ordenação de bytes

Ordenação de bytes

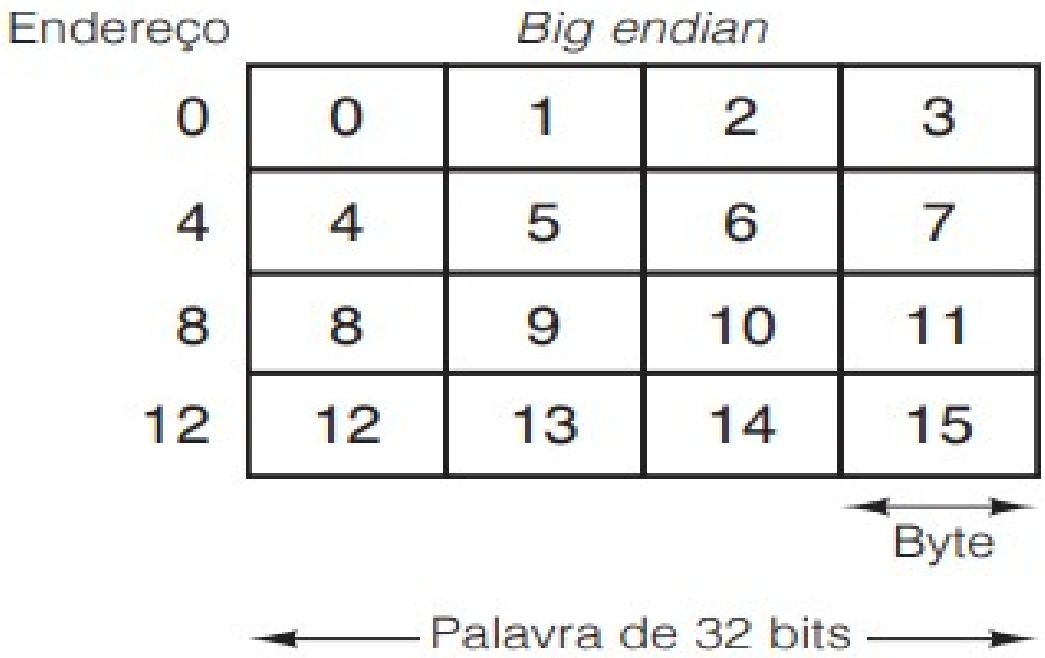
Como você acha que os bytes são organizados na memória?

Ordenação de bytes

- Os bytes em uma palavra podem ser numerados da esquerda para a direita ou da direita para a esquerda.

Ordenação de bytes

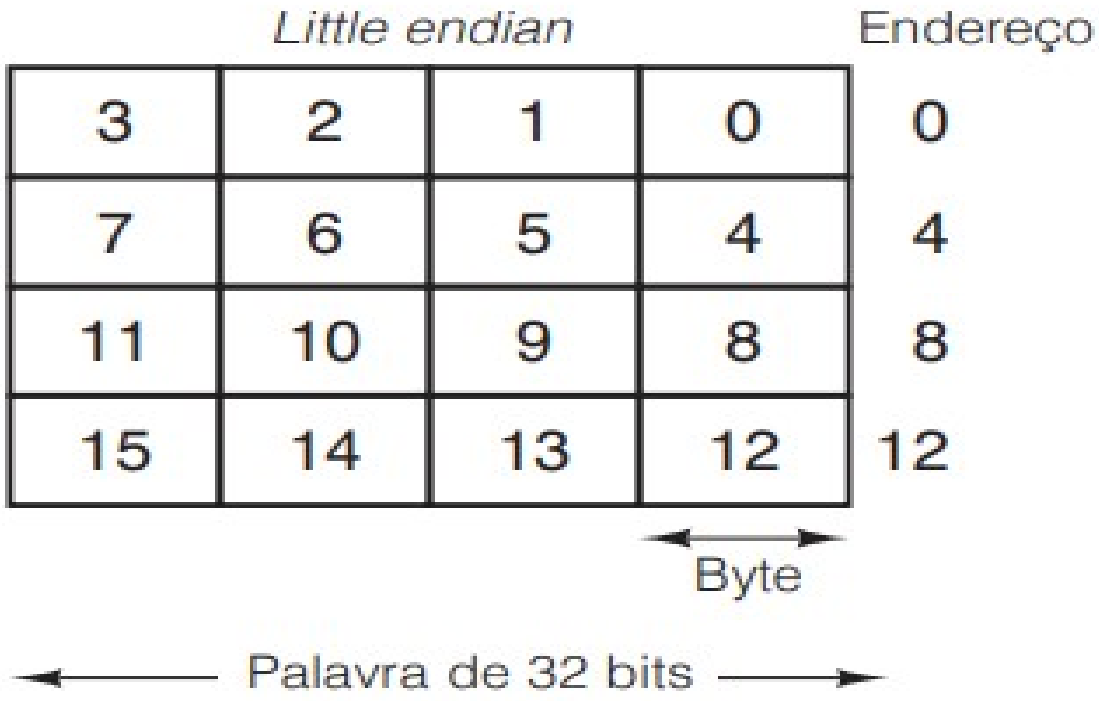
- A Figura 2.11(a) retrata parte da memória de um computador de 32 bits cujos bytes são numerados **da esquerda para a direita**, tal como o SPARC ou os grandes mainframes da IBM.



(a)

Ordenação de bytes

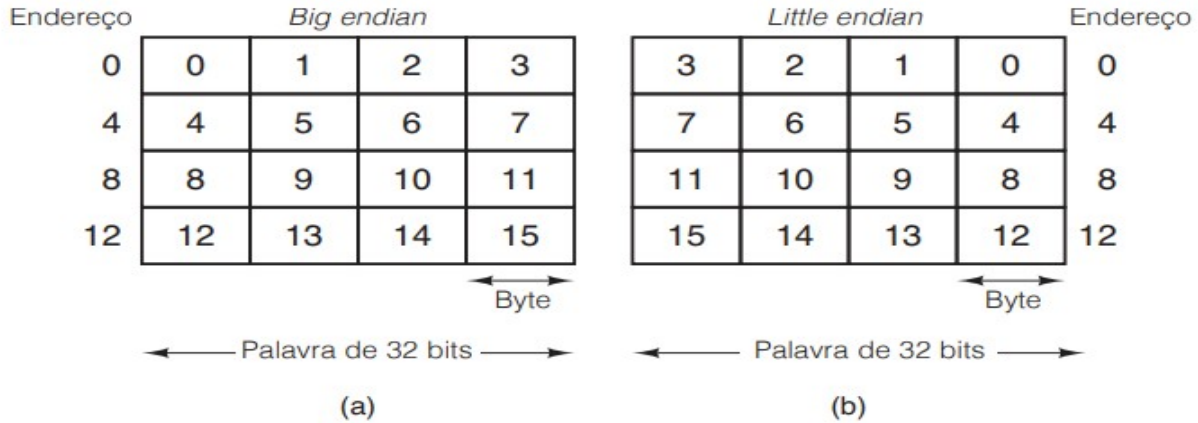
- A Figura 2.11(b) dá uma representação análoga de um computador de 32 bits que usa uma numeração **da direita para a esquerda**, como a família Intel.



(b)

Ordenação de bytes

Figura 2.11 (a) Memória *big endian*. (b) Memória *little endian*.



- Ex:
- É importante entender que, tanto no sistema *big endian* como no *little endian*, um inteiro de 32 bits com o valor decimal de 6 é representado pelos bits 110
- No esquema *big endian*, os bits 110 estão no byte 3 (ou 7, ou 11 etc.),
- Enquanto no esquema *little endian* eles estão no byte 0 (ou 4, ou 8 etc.).

Ordenação de bytes

Haveria algum problema nessa organização de memória?

Ordenação de bytes

- Se os computadores somente armazenassem inteiros, não haveria nenhum problema.

- Contudo, muitas aplicações requerem uma mistura de inteiros, cadeias de caracteres e outros tipos de dados.

Ordenação de bytes

- Ex: um simples registro de pessoal composto de:
  - uma cadeia (nome do empregado) e
  - dois inteiros (idade e número do departamento).
- A cadeia é encerrada com 1 ou mais bytes de valor 0 para completar uma palavra.
- Para o registro "Jim Smith, idade 21, departamento 260
  - (1 × 256 + 4 = 260 -> lembre de que podemos só armazenar 2^8 bits)
- A representação big endian é mostrada na Figura 2.12(a) e a representação little endian é mostrada na Figura 2.12(b).

Ordenação de bytes

Figura 2.12 (a) Registro de pessoal para uma máquina *big endian*. (b) O mesmo registro para uma máquina *little endian*. (c) Resultado da transferência do registro de uma máquina *big endian* para uma *little endian*. (d) Resultado da troca de bytes (c).

Big endian

0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(a)

Little endian

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
0	0	0	21	12
0	0	1	4	16

(b)

Transferência de *big endian* para *little endian*

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
21	0	0	0	12
4	1	0	0	16

(c)

Transferência e troca

J	I	M		0
S	M	I	T	4
H	0	0	0	8
0	0	0	21	12
0	0	1	4	16

(d)

Ordenação de bytes

- Os problemas começam quando uma das máquinas tenta enviar um registro à outra por uma rede.
- Vamos supor que **a big endian envie o registro à little endian** um byte por vez, começando com o byte 0 e terminando com o byte 19.

Figura 2.12 (a) Registro de pessoal para uma máquina *big endian*. (b) O mesmo registro para uma máquina *little endian*. (c) Resultado da transferência do registro de uma máquina *big endian* para uma *little endian*. (d) Resultado da troca de bytes (c).

Big endian

0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(a)

Little endian

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
0	0	0	21	12
0	0	1	4	16

(b)

Transferência de *big endian* para *little endian*

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
21	0	0	0	12
4	1	0	0	16

(c)

Transferência e troca

J	I	M		0
S	M	I	T	4
H	0	0	0	8
0	0	0	21	12
0	0	1	4	16

(d)

Ordenação de bytes

- Sendo otimistas que os bits dos bytes não sejam invertidos pela transmissão.
- Portanto, o byte 0 da big endian entra na memória da little endian no byte 0 e assim por diante, como mostra a Figura 2.12(c).

Figura 2.12 (a) Registro de pessoal para uma máquina *big endian*. (b) O mesmo registro para uma máquina *little endian*. (c) Resultado da transferência do registro de uma máquina *big endian* para uma *little endian*. (d) Resultado da troca de bytes (c).

Big endian

0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(a)

Little endian

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
0	0	0	21	12
0	0	1	4	16

(b)

Transferência de *big endian* para *little endian*

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
21	0	0	0	12
4	1	0	0	16

(c)

Transferência e troca

J	I	M		0
S	M	I	T	4
H	0	0	0	8
0	0	0	21	12
0	0	1	4	16

(d)

Ordenação de bytes

- Quando a little endian tenta imprimir o nome, ela funciona bem, mas a idade e o departamento também fica errado.
- Essa situação surge porque **a transmissão inverteu a ordem dos caracteres em uma palavra**, como deveria, mas **também inverteu os bytes de um inteiro**, o que não deveria.

Figura 2.12 (a) Registro de pessoal para uma máquina *big endian*. (b) O mesmo registro para uma máquina *little endian*. (c) Resultado da transferência do registro de uma máquina *big endian* para uma *little endian*. (d) Resultado da troca de bytes (c).

Big endian

0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(a)

Little endian

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
0	0	0	21	12
0	0	1	4	16

(b)

Transferência de *big endian* para *little endian*

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
21	0	0	0	12
4	1	0	0	16

(c)

Transferência e troca

J	I	M		0
S	M	I	T	4
H	0	0	0	8
0	0	0	21	12
0	0	1	4	16

(d)

Ordenação de bytes

- Uma solução p/ os inteiros é fazer o software **inverter os bytes de uma palavra após tê-la copiado.**
- Isso leva à Figura 2.12(d), que transfere os dois inteiros corretamente, mas transforma a cadeia em "MIJTIMS" e deixa o "H" perdido no meio do nada.
- Essa inversão da cadeia ocorre porque, ao ler a cadeia, **o computador lê primeiro o byte 0** (um espaço), **em seguida o byte 1** (**M**), **e assim por diante**.

Figura 2.12 (a) Registro de pessoal para uma máquina *big endian*. (b) O mesmo registro para uma máquina *little endian*. (c) Resultado da transferência do registro de uma máquina *big endian* para uma *little endian*. (d) Resultado da troca de bytes (c).

Big endian

0	J	I	M	
4	S	M	I	T
8	H	0	0	0
12	0	0	0	21
16	0	0	1	4

(a)

Little endian

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
0	0	0	21	12
0	0	1	4	16

(b)

Transferência de *big endian* para *little endian*

	M	I	J	0
T	I	M	S	4
0	0	0	H	8
21	0	0	0	12
4	1	0	0	16

(c)

Transferência e troca

J	I	M		0
S	M	I	T	4
H	0	0	0	8
0	0	0	21	12
0	0	1	4	16

(d)

Ordenação de bytes

- Não há nenhuma solução simples.
- Um modo que funciona – porém, ineficiente – é incluir um cabeçalho na frente de cada item de dado, que informa **qual tipo de dado** vem a seguir (cadeia, inteiro ou outro) e qual é **seu comprimento**.
- Isso permite que o destinatário efetue apenas as conversões necessárias.
- **É preciso deixar claro que a falta de um padrão para a ordenação de bytes é um grande aborrecimento quando há troca de dados entre máquinas diferentes.**

Códigos de correção de erro

DETECÇÃO DE ERROS PELO MÉTODO DE PARIDADE

- Dadas duas palavras de código quaisquer, por exemplo, 10001001 e 10110001, é possível determinar quantos bits correspondentes são diferentes.
    - Nesse caso, 3 bits são diferentes.
- Para saber quantos bits são diferentes, basta calcular o **OU EXCLUSIVO booleano bit por bit** das duas palavras de código e contar o número de bits 1 no resultado.

DETECÇÃO DE ERROS PELO MÉTODO DE PARIDADE

- O número de posições de bit nas quais as duas palavras de código diferem é denominado **distância de Hamming**.
- Sua importância é que, se duas palavras de código estiverem separadas por uma distância de Hamming d, será preciso d erros de único bit para converter uma na outra.
- Por exemplo, as palavras de código 11110001 e 00110000 estão **a uma distância de Hamming 3 porque é preciso 3 erros de único bit para converter uma na outra.**

DETECÇÃO DE ERROS PELO MÉTODO DE PARIDADE

- A **movimentação de dados** e **códigos binários** de um local para outro é a **operação** mais **frequentemente realizada** em sistemas.
- Exemplos:

- A **transmissão de voz** digitalizada por um enlace.
- O **armazenamento e a recuperação de dados** em dispositivos externos.
- A **transmissão de dados digitais** de um computador para outro.

DETECÇÃO DE ERROS PELO MÉTODO DE PARIDADE

- Quando uma informação é transmitida de um dispositivo (transmissor) para outro (receptor), há a possibilidade de **ocorrência de erro**
  - quando o receptor não recebe uma informação idêntica àquela enviada pelo transmissor.
- A principal causa de erro de transmissão é o **ruído elétrico**.
  - O transmissor envia um sinal, no formato serial por meio de uma linha para o receptor. Entretanto, no momento em que o sinal chega ao receptor, apresenta ruído sobreposto ao sinal original.

DETECÇÃO DE ERROS PELO MÉTODO DE PARIDADE

- Às vezes o ruído tem **amplitude grande** o suficiente para alterar o nível lógico do sinal, como ocorre no **ponto x.**
- Quando isso ocorre, o receptor pode interpretar incorretamente que o bit em questão tenha nível lógico 1, o que não corresponde à informação enviada pelo transmissor.

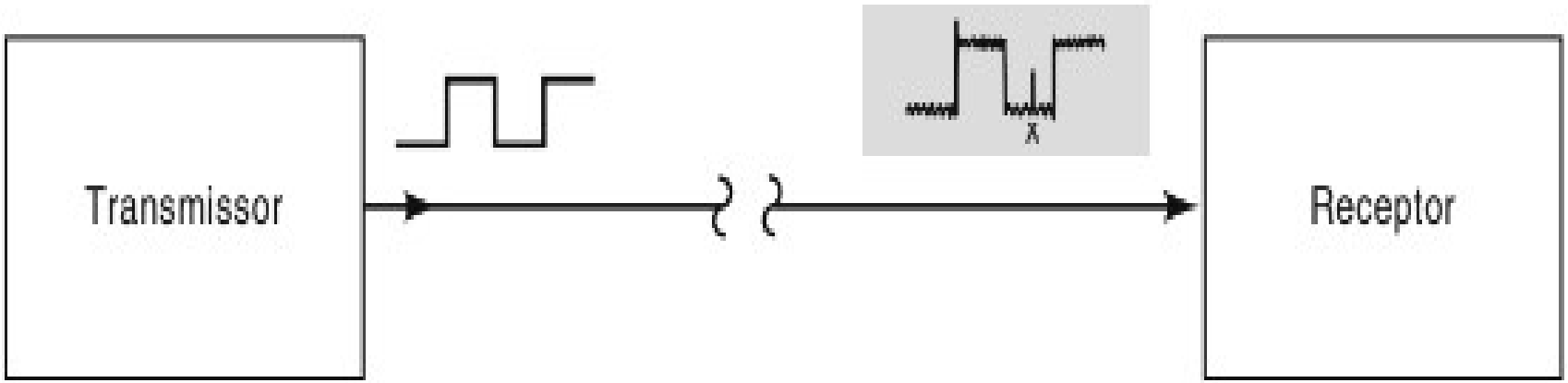


FIGURA 2.4 Exemplo de um erro causado por um ruído em uma transmissão digital.

Bit de Paridade

- Muitos sistemas utilizam algum método de detecção (e algumas vezes de correção) de erros.
- Uma das técnicas mais simples e mais usadas para detecção de erros é o **método de paridade**.
- Um bit de paridade consiste em um **bit extra** anexado ao conjunto de bits do código a ser transferido de uma localidade para outra.
- O bit de paridade pode ser 0 ou 1, **dependendo do número de 1s contido no conjunto de bits do código.**

Bit de Paridade

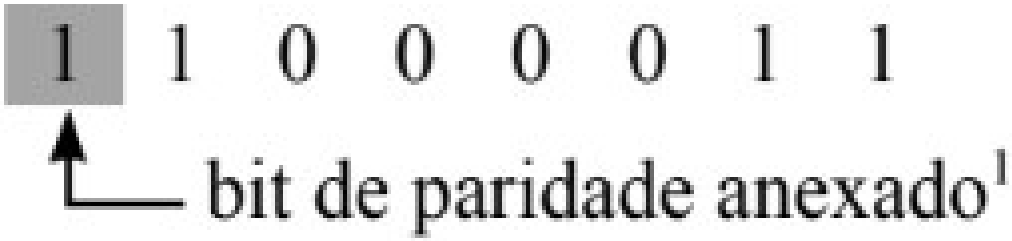
- Dois métodos diferentes:
  - **No método que usa paridade par**, o valor do bit de paridade é determinado para que **o número total de 1s** no conjunto de bits do código (incluindo o bit de paridade) **seja par**.

Por exemplo:

Suponha que o conjunto de bits seja 1000011.

Esse é o código ASCII do caractere 'C'.

Esse conjunto de bits tem **três 1s**; portanto, anexamos **um bit de paridade par igual a 1 para tornar par o número total de 1s**. O novo conjunto de bits, incluindo o bit de paridade, passa a ser:



Bit de Paridade

- **Se o grupo de bits do código contiver um número par de 1s, o bit de paridade terá o valor 0.**
- Por exemplo, se o conjunto de bits do código fosse **1000001** (o código ASCII para 'A'), o bit de paridade designado seria o 0, de modo que o novo código, incluindo o bit de paridade, seria **01000001**.

Bit de Paridade

- **O método de paridade ímpar** é usado exatamente da mesma maneira, exceto que o bit de paridade é determinado para que **o número total de 1s**, incluindo o bit de paridade, seja **ímpar**.
- Por exemplo, para o grupo de bits 1000001, deve ser 1,
- e para o grupo de bits 1000011, deve ser 0.
- Quer a paridade utilizada seja par, quer seja ímpar, **o bit de paridade passa a ser parte do código atual da informação.**
- Por exemplo, anexando um bit de paridade ao código ASCII de 7 bits, geramos um código de 8 bits.
- Assim, o bit de paridade é tratado exatamente como qualquer outro bit do código.

Bit de Paridade

- O bit de paridade é gerado para **detectar erros de um só bit** que ocorram durante a transmissão de um código de um local para outro.
- Por exemplo, suponha que o caractere 'A' seja transmitido e que seja usada a **paridade ímpar**. O código transmitido seria:

1 1 0 0 0 0 0 1

Bit de Paridade

- Quando ele chega ao circuito receptor, o receptor verifica se o código contém um número ímpar de 1s (incluindo o bit de paridade).
- Em caso afirmativo, o receptor considera que o código foi recebido corretamente.
- No entanto, suponha que, devido a algum ruído ou mau funcionamento do circuito receptor, seja recebido o seguinte código:

1 1 0 0 0 0 0 0

Bit de Paridade

No entanto, suponha que, devido a algum ruído ou mau funcionamento do circuito receptor, seja recebido o seguinte código:

O que você acha que aconteceria?

1 1 0 0 0 0 0 0

Bit de Paridade

- O receptor identificará que o código tem um número par de 1s.
- Isso significa, para o receptor, que há um erro no código,
    - presumindo que transmissor e receptor tenham usado paridade ímpar.
- Entretanto, não há como o receptor identificar qual bit está errado
    - visto que ele não sabe qual é o código correto.
- **É evidente que o método de paridade não funcionará se ocorrer erro em dois bits, porque dois bits errados não geram alteração na paridade do código.**
- **Na prática, o método de paridade é usado apenas nas situações em que a probabilidade de erro de um único bit é baixa e, em dois bits, é essencialmente zero.**

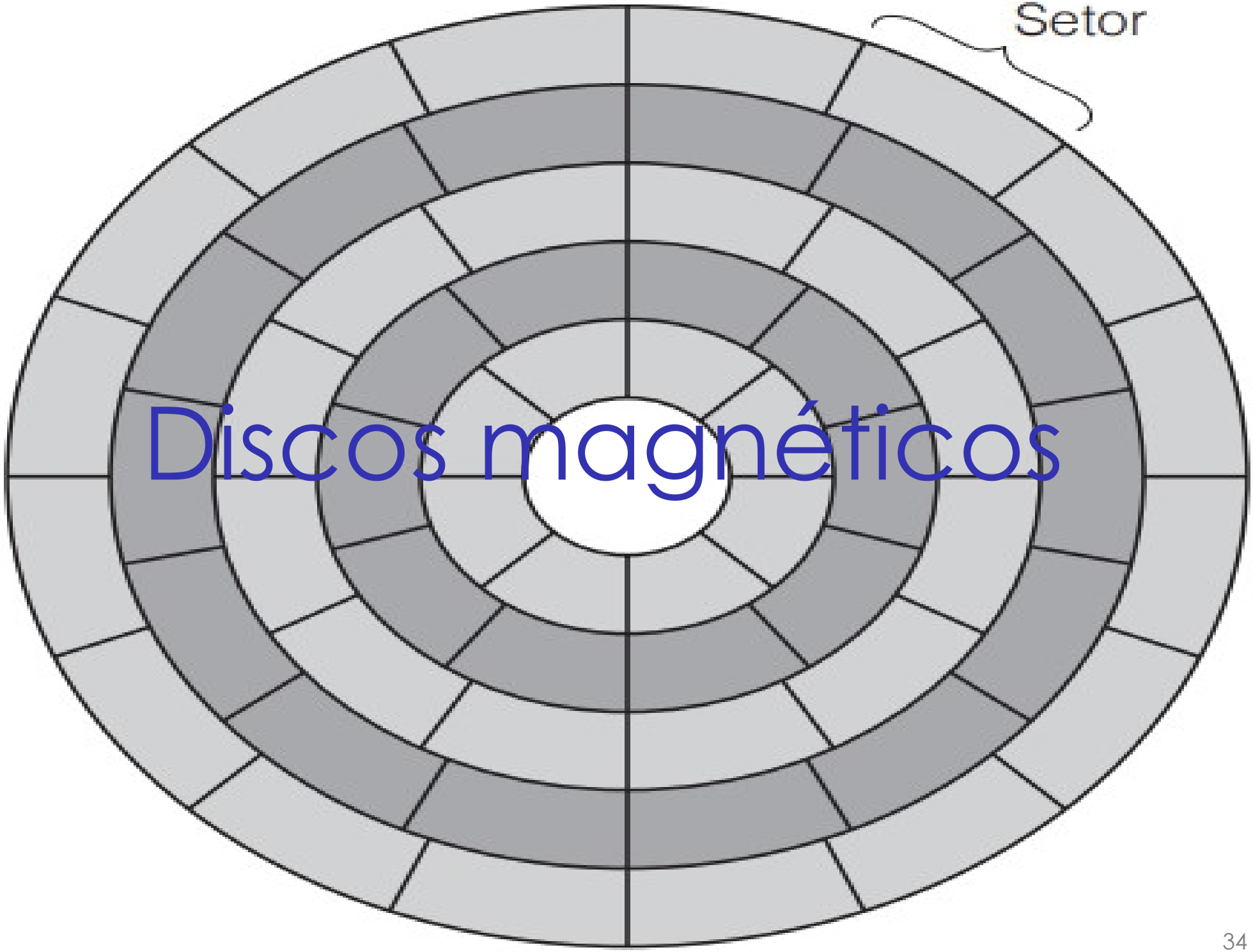
Bit de Paridade

- Quando se usa o método de paridade, tem de haver uma **concordância** entre transmissor e receptor em relação ao tipo de paridade (par ou ímpar) a ser usada.
- Não exista vantagem de um método sobre o outro,
    - Porém a paridade par é mais usada.
- O transmissor anexa um bit de paridade a cada unidade de informação transmitida.
    - Por exemplo, se o transmissor está enviando um dado codificado em ASCII, ele anexará um bit de paridade a cada conjunto ASCII de 7 bits.

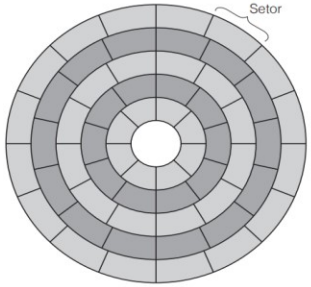
Bit de Paridade

- Quando o receptor analisar o dado recebido, ele verificará se a quantidade de 1s de cada conjunto de bits (incluindo o bit de paridade) está de acordo com o método de paridade escolhido previamente.
- Essa operação é frequentemente denominada **verificação de paridade dos dados.**
- Quando um erro for detectado, o receptor poderá enviar uma mensagem de volta ao transmissor, solicitando a retransmissão do último conjunto de dados.

Discos magnéticos



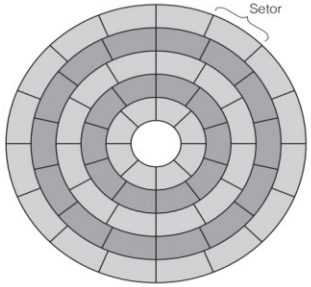
Discos magnéticos



ESTRUTURA DE UM DISCO MAGNÉTICO

- **Disco rígido** (do inglês ***Hard Disk***) é a parte do computador onde são armazenadas as informações, ou seja, é a "memória permanente" propriamente dita (não confundir com "memória RAM").
- É caracterizado como memória física, não-volátil, que é aquela na qual as informações não são perdidas quando o computador é desligado.
- O disco rígido é um sistema lacrado contendo discos de metal recobertos por material magnético onde os dados são gravados através de cabeças, e revestido externamente por uma proteção metálica que é presa ao gabinete do computador por parafusos.

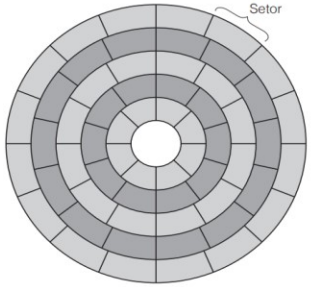
Discos magnéticos



ESTRUTURA DE UM DISCO MAGNÉTICO

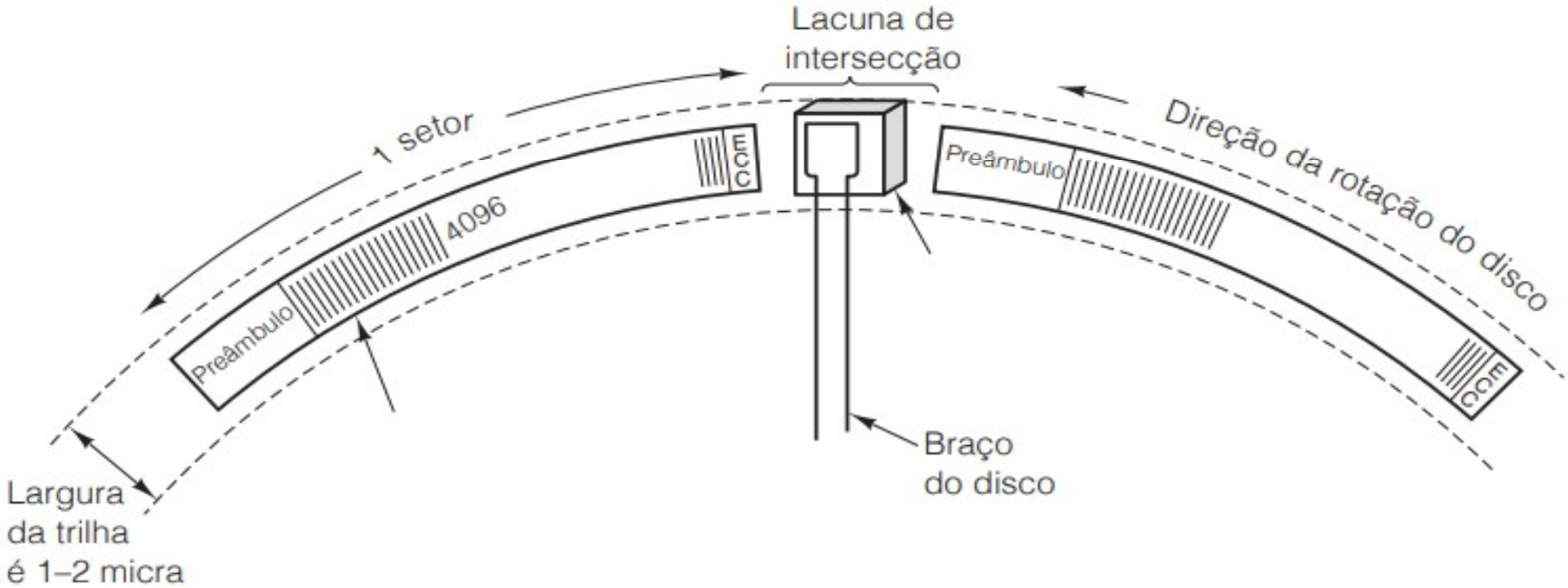
- É nele que normalmente gravamos dados (informações) e a partir dele lançamos e executamos nossos programas mais usados.
- Este sistema é necessário porque o conteúdo da memória RAM é apagado quando o computador é desligado.
- Desta forma, temos um meio de executar novamente programas e carregar arquivos contendo os dados da próxima vez em que o computador for ligado.
- O disco rígido é também chamado de memória de massa ou ainda de memória secundária.

Discos magnéticos

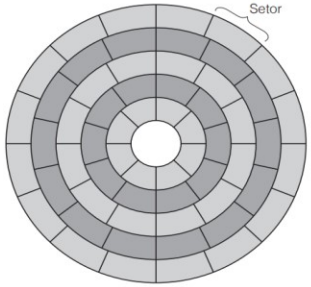


- Um disco magnético é composto de um ou mais pratos de alumínio com um revestimento magnetizável.
- Tem normalmente de 3 a 9 cm.
- Um cabeçote de disco flutua logo acima da superfície.

Figura 2.19 Porção de uma trilha de disco. Dois setores são ilustrados.

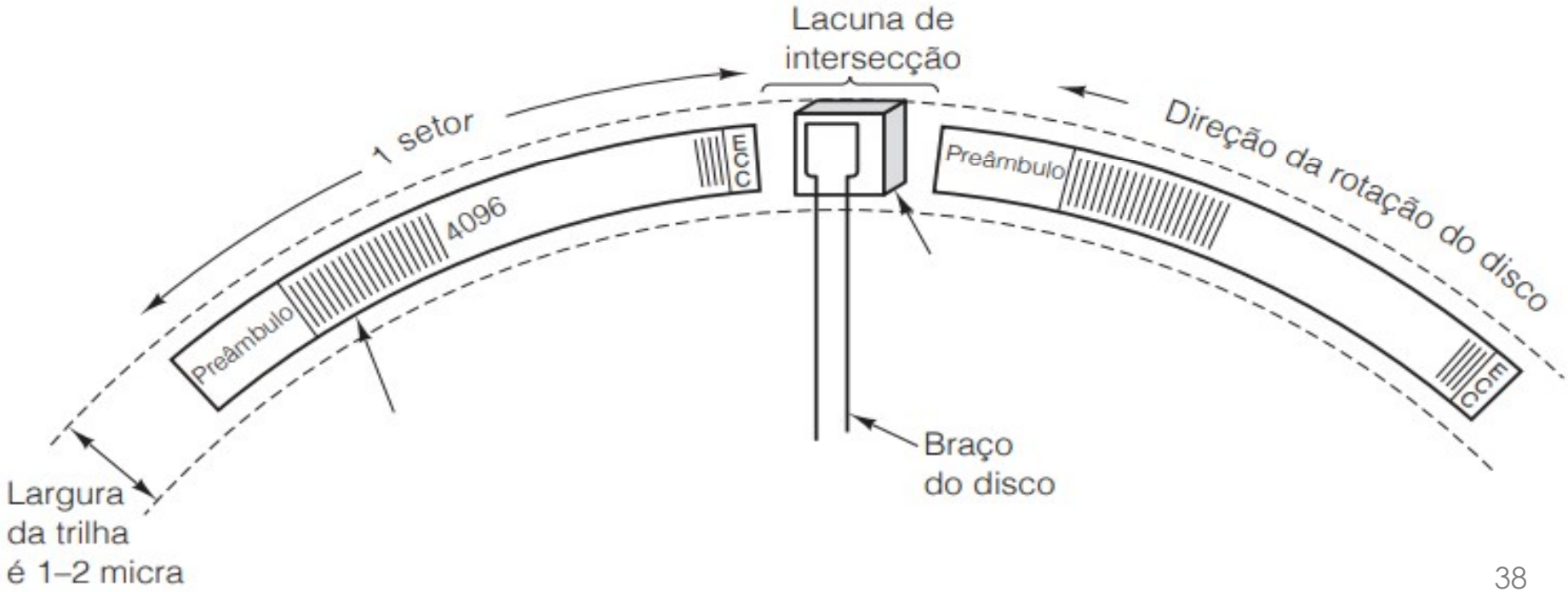


Discos magnéticos

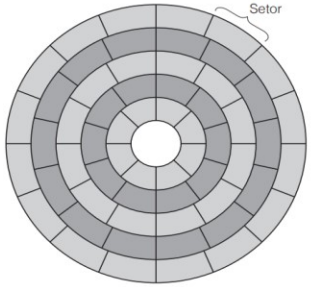


- Quando uma corrente positiva ou negativa passa pelo cabeçote, ele alinha as partículas magnéticas para a esquerda ou para a direita, dependendo da polaridade da corrente.

Figura 2.19 Porção de uma trilha de disco. Dois setores são ilustrados.

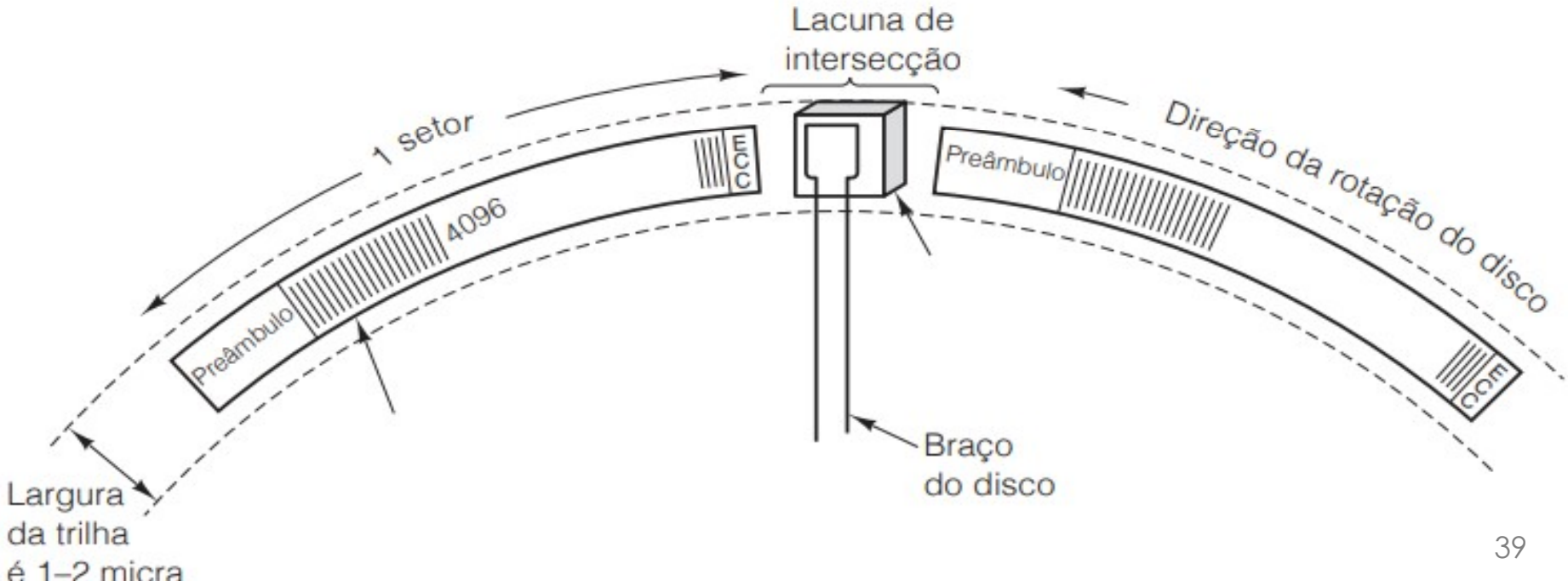


Discos magnéticos

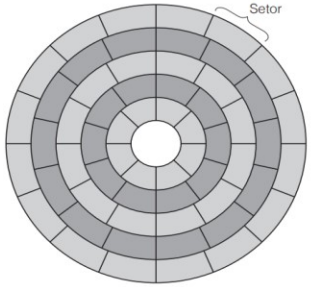


- Quando o cabeçote passa sobre uma área magnetizada, uma corrente positiva ou negativa é induzida nele, o que possibilita **a leitura dos bits armazenados antes.**
- Assim, à medida que o prato gira sob o cabeçote, **uma corrente de bits pode ser escrita e mais tarde lida.**

Figura 2.19 Porção de uma trilha de disco. Dois setores são ilustrados.

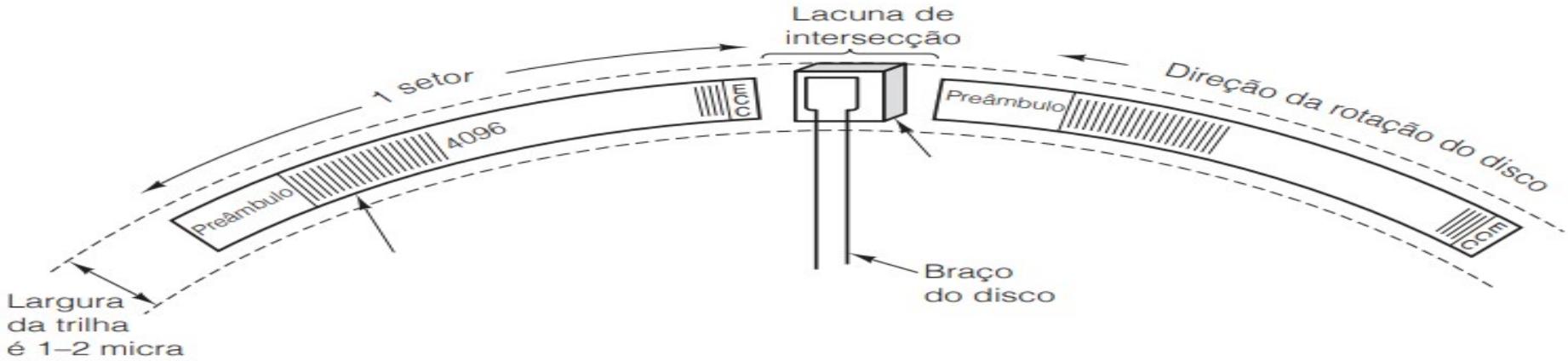


Discos magnéticos

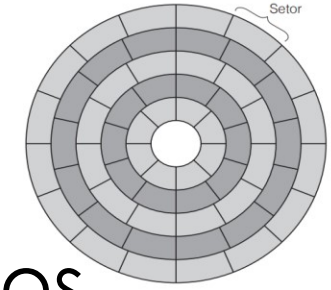


- Todos os discos têm braços móveis que podem se mover para dentro e para fora a diferentes distâncias da haste ao redor da qual o prato gira.
- A cada distância pode ser escrita uma trilha diferente.
- As trilhas são uma série de círculos concêntricos ao redor da haste.

Figura 2.19 Porção de uma trilha de disco. Dois setores são ilustrados.

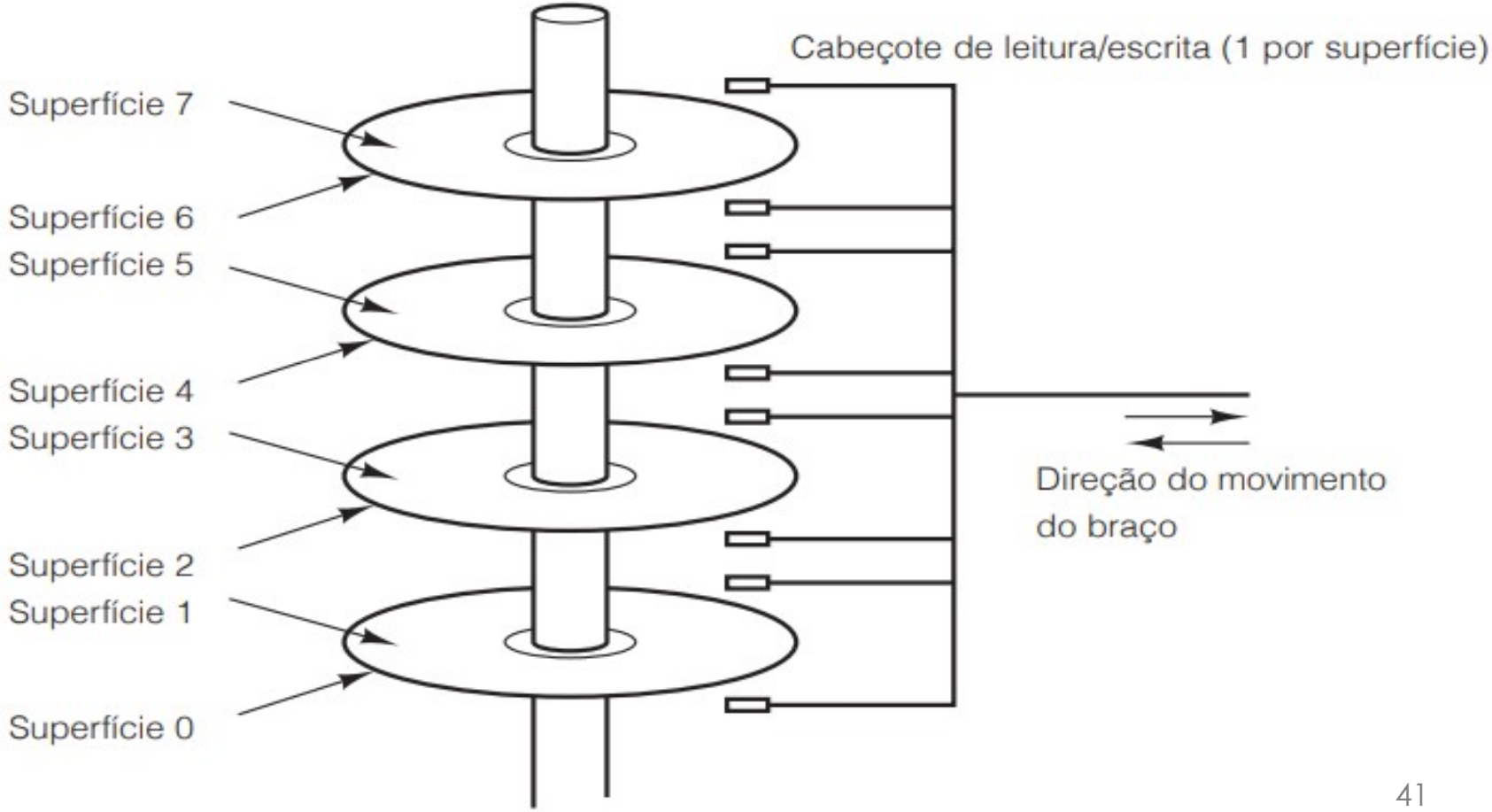


Discos magnéticos

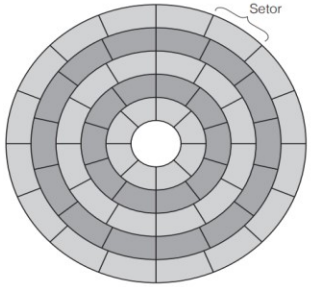


- A maioria dos discos é composta de vários pratos empilhados na vertical, como ilustrado na Figura 2.20.

Figura 2.20 Disco com quatro pratos.

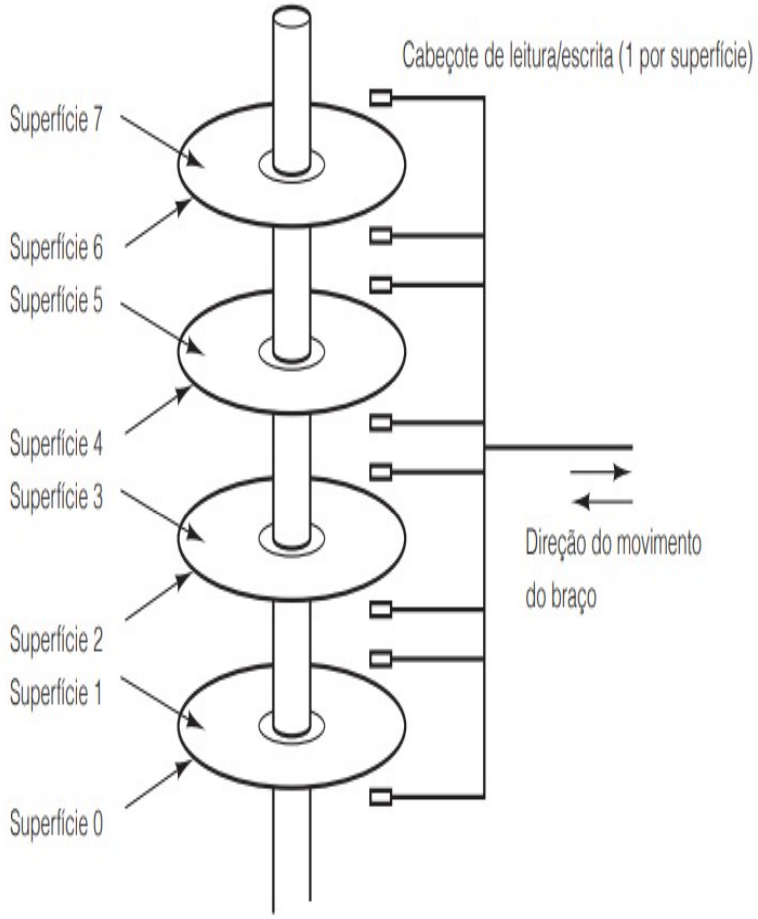


Discos magnéticos

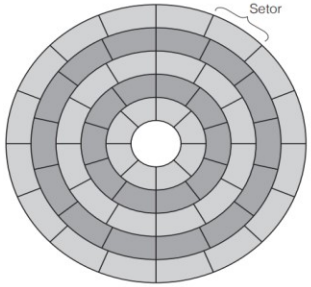


- Cada superfície tem seu próprio braço e cabeçote.
- Os braços são agrupados de modo que todos se movimentem para diferentes posições ao mesmo tempo.
- O conjunto de trilhas em uma dada posição é denominado cilindro.

Figura 2.20 Disco com quatro pratos.

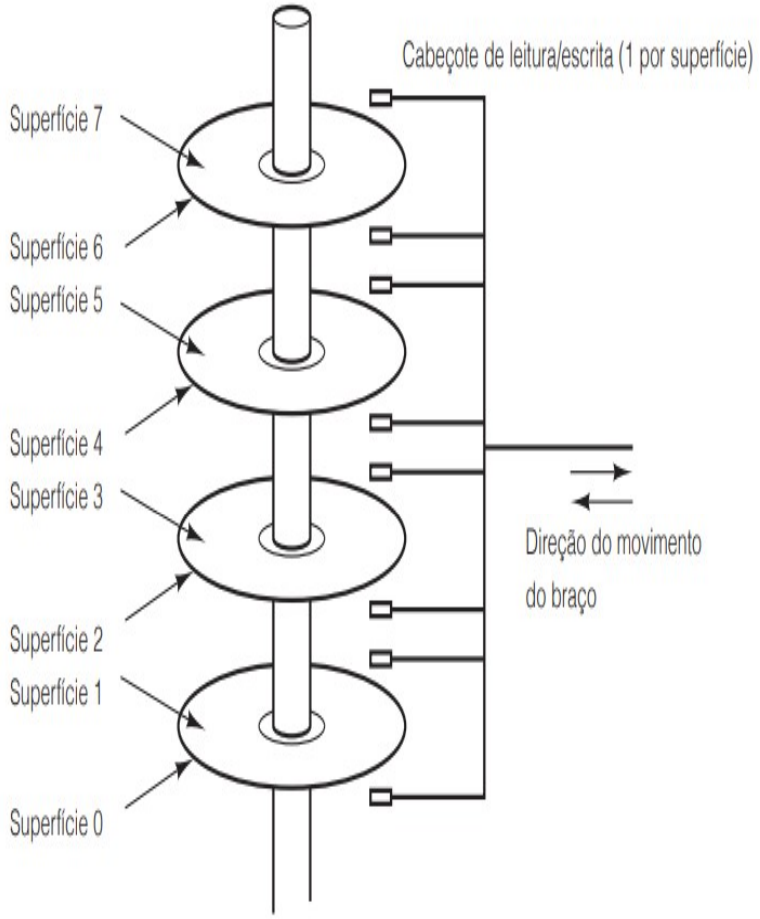


Discos magnéticos

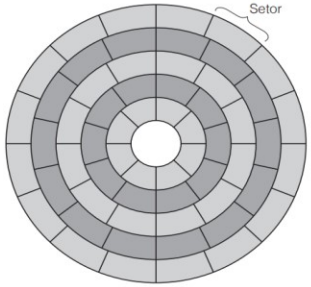


- Os discos usados hoje em PCs costumam ter de 1 a 12 pratos,
- Resultando em 2 a 24 superfícies de gravação.
- Discos de última geração podem armazenar 1 TB em um único prato, e esse limite certamente crescerá com o tempo.

Figura 2.20 Disco com quatro pratos.

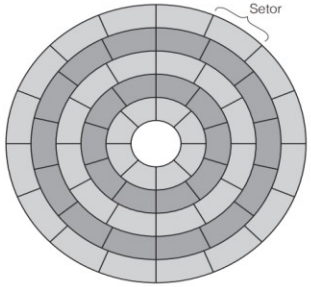


Discos magnéticos



- O desempenho do disco depende de vários fatores:
- Para ler ou escrever um setor, primeiro o braço deve se deslocar até a posição correta.
- Essa ação é denominada busca (seek).
- Tempos médios de busca (entre trilhas **aleatórias**) estão na **faixa de 5 a 10 ms**, embora buscas entre trilhas **consecutivas** estão abaixo de **1 ms**.

Discos magnéticos

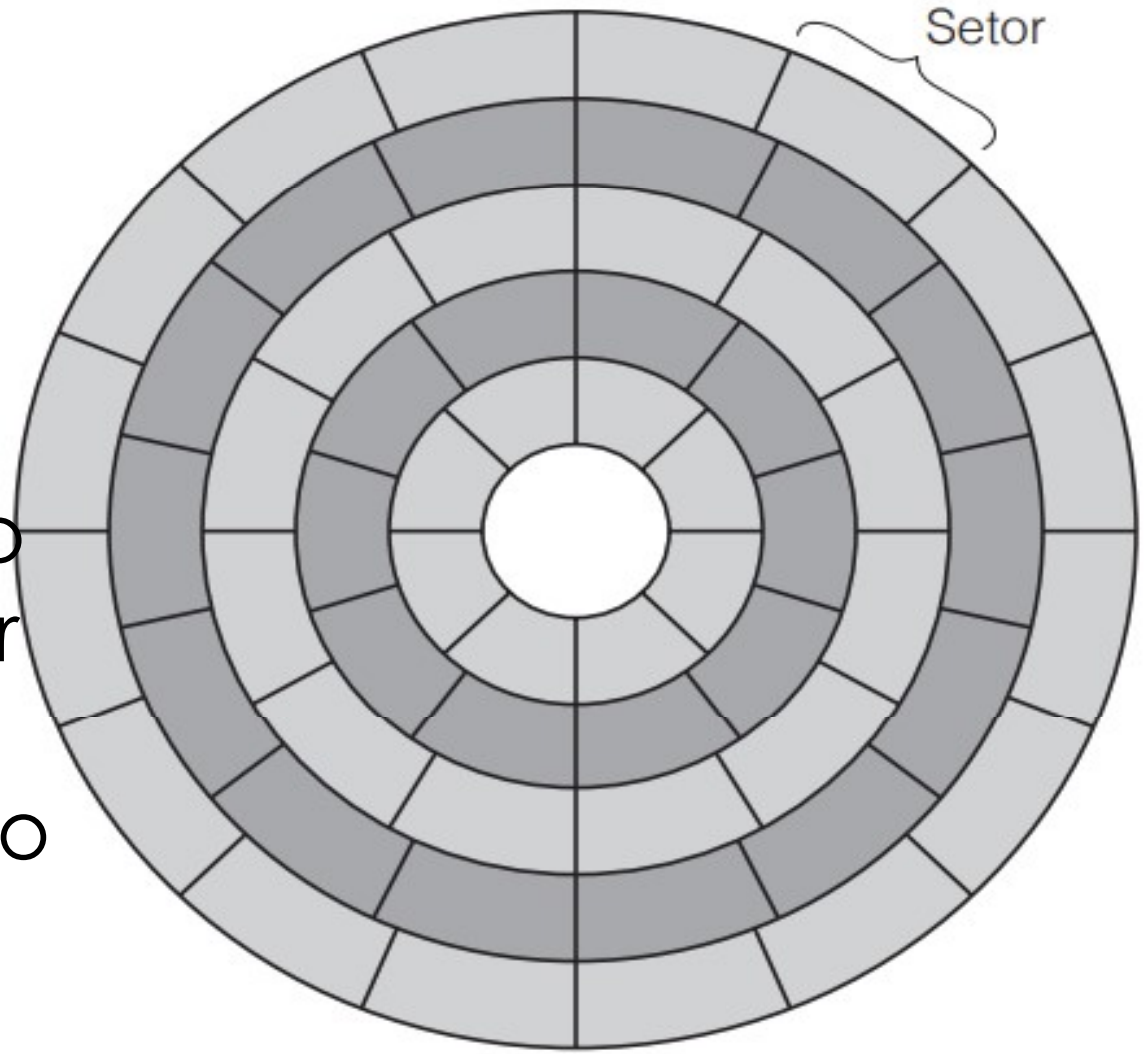


- O desempenho do disco depende de vários fatores:
- Latência rotacional: tempo para que o inicio do bloco que contenha o registro a ser lido passe pelo cabeçote de leitura/gravação

- Tempo de transferência: quantidade de tempo necessário para que um bloco (ou setor) seja transferido para um *buffer* de memória

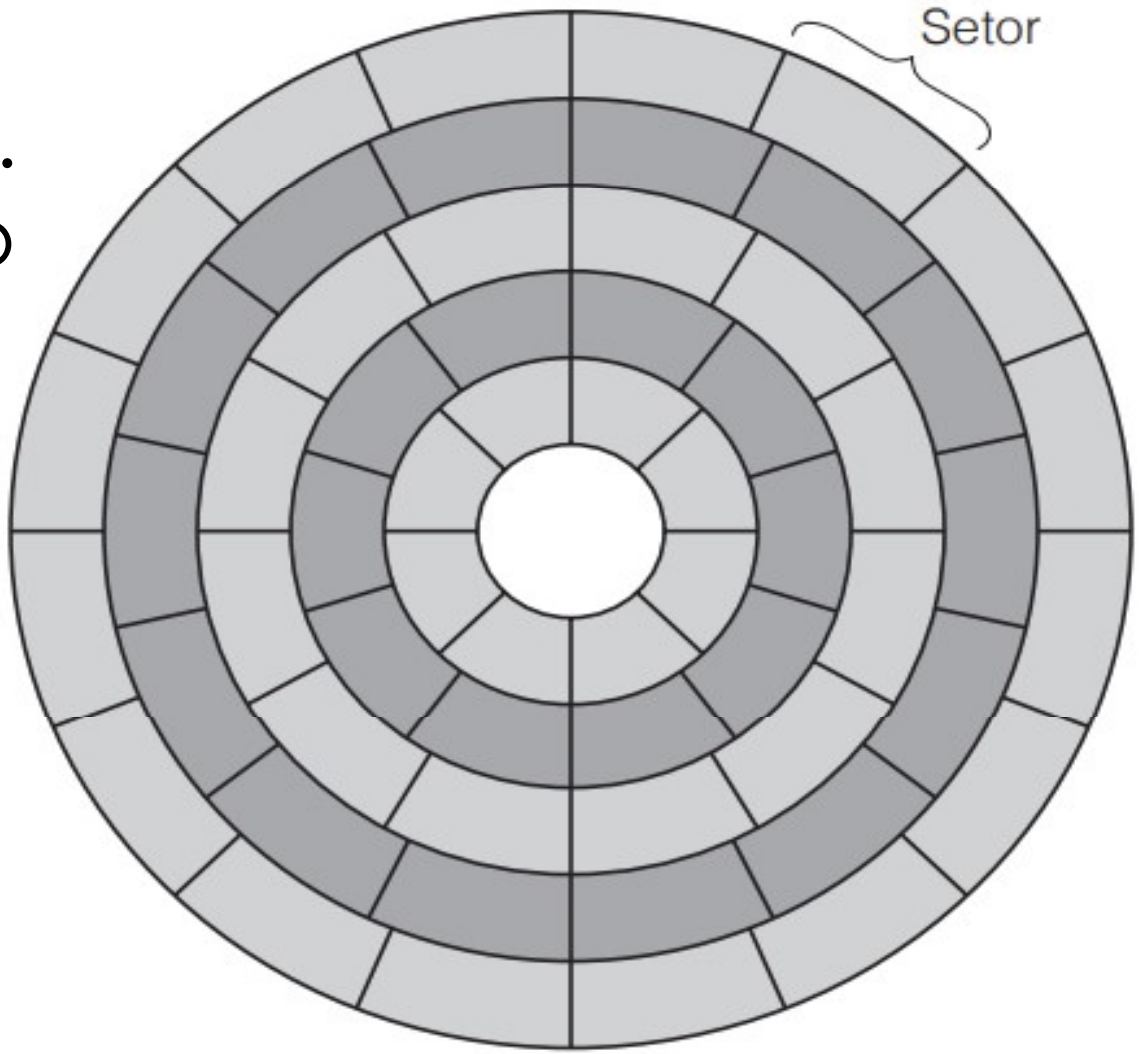
Discos magnéticos

- A distância ao redor das trilhas mais externas é maior do que a das trilhas mais internas.
- Os cilindros são divididos em zonas e o número de setores por trilha aumenta de zona em zona partindo da trilha mais interna para a mais externa.

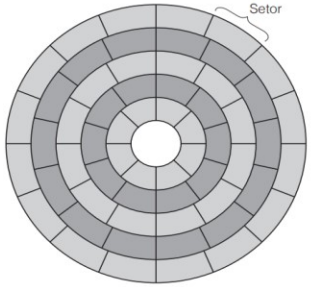


Discos magnéticos

- A divisão por zonas aumenta a capacidade do disco.
- Todos os setores são do mesmo tamanho.
- A Figura mostra um disco com cinco zonas.

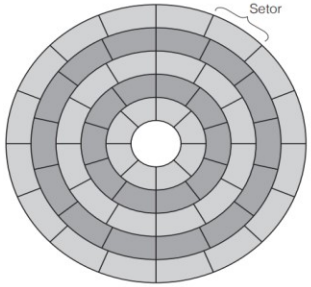


Discos magnéticos



- Associado a cada disco há um **controlador de disco**, um chip que controla o disco.
- Entre as tarefas do controlador estão:
  - aceitar comandos do software, como READ, WRITE e FORMAT (escrevendo todos os preâmbulos),
  - controlar o movimento do braço,
  - detectar e corrigir erros
  - e converter bytes de 8 bits lidos na memória em uma corrente serial de bits e vice-versa.

Discos magnéticos



- Alguns controladores também manipulam o buffer de múltiplos setores, fazendo cache de setores lidos para potencial uso futuro e remapeando setores ruins.
- Quando descobre um setor ruim, o controlador o substitui por um dos setores reservados para esse fim dentro de cada cilindro ou zona.

Referência

- TANENBAUM, Andrew S.; ZUCCHI, Wagner Luiz. **Organização estruturada de computadores**. Pearson Prentice Hall, 2013.

- TOCCI, Ronald J.; WIDMER, Neal S.; MOSS, Gregory L.. **Sistemas digitais:** princípios e aplicações. 11. ed. São Paulo : Pearson Prentice Hall, 2011.

- Discos rígidos: https://www.gta.ufrj.br/grad/07_1/hd/func.html